

Frontend–Blockchain State Desynchronization After Successful NFT Mint

Prepared by	JERUZSALEM – DAC Infra Tester
Date	Sunday, 10 May 2026
Environment	WSL Ubuntu Node (RPC :8546) + Rabby Wallet — Indonesia / CGNAT
Report Type	Application Layer Consistency Investigation — Indexer / Frontend Synchronization
Relates to	EIP-1559 Compatibility & Txpool Analysis (09 May 2026)

Executive Summary

This report documents an application-layer state desynchronization issue observed on DAC Testnet following a successful NFT badge mint transaction. Although the mint transaction was finalized on-chain and confirmed through the DAC blockchain explorer — with the NFT verifiably present in the wallet — the frontend dashboard continued to display the badge as unminted, with the mint button remaining active.

The findings indicate a synchronization delay or failure between blockchain state, backend indexer infrastructure, and frontend UI rendering. This creates a misleading user experience where on-chain reality is not reflected in the application interface.

Layer	Status	Detail
Blockchain / On-chain layer	Operational	TX finalized, NFT confirmed
DAC Blockchain Explorer	Accurate	NFT ownership visible
Frontend dashboard (badge status)	Desynchronized	Still showing unminted
Backend indexer / event listener	Delayed / Failed	Mint event not processed

Environment

Node	WSL Ubuntu — local dacnode binary
RPC Endpoint	127.0.0.1:8546 (local node) · rpctest.dachain.tech (official)
Wallet	Rabby Wallet
Wallet Address	0x870ad63acc507cdfd878F170606d19ae78988AFE
NFT Contract	0xB36ab4c2Bd6aCfC36e9D6c53F39F4301901Bd647
Target	DAC Inception Badge System
Network	Indonesia — Residential CGNAT

Observed Behavior

After submitting the NFT badge mint transaction, the following state was observed across different layers of the DAC infrastructure:

Component	Status	Notes
Transaction submission	Success	Submitted via node
On-chain TX finalization	Confirmed	Verified via explorer
NFT ownership on-chain	Confirmed	5 NFTs in wallet
Explorer badge verification	Accurate	NFT visible on-chain
Frontend badge status	Stale	Still showing unminted
Mint button UI state	Incorrect	Remains active post-mint
UI synchronization	Failed	No automatic refresh

Transaction Evidence

Node Log — Mint Sequence

The following log entries were captured from the WSL node at the time of the mint transaction. The sequence shows `eth_estimateGas` initially reverting (simulation failure), followed by successful transaction submission — confirming the transaction reached the network.

[illegible]

The `eth_estimateGas` revert is a simulation failure — not a transaction failure. The custom Solidity error selector (`0xf3998cd1`) returned in `errdata` indicates the contract has anti-simulation logic or strict precondition checks that fire during gas estimation. The transaction itself was submitted and finalized independently of the simulation result.

Transaction Hash

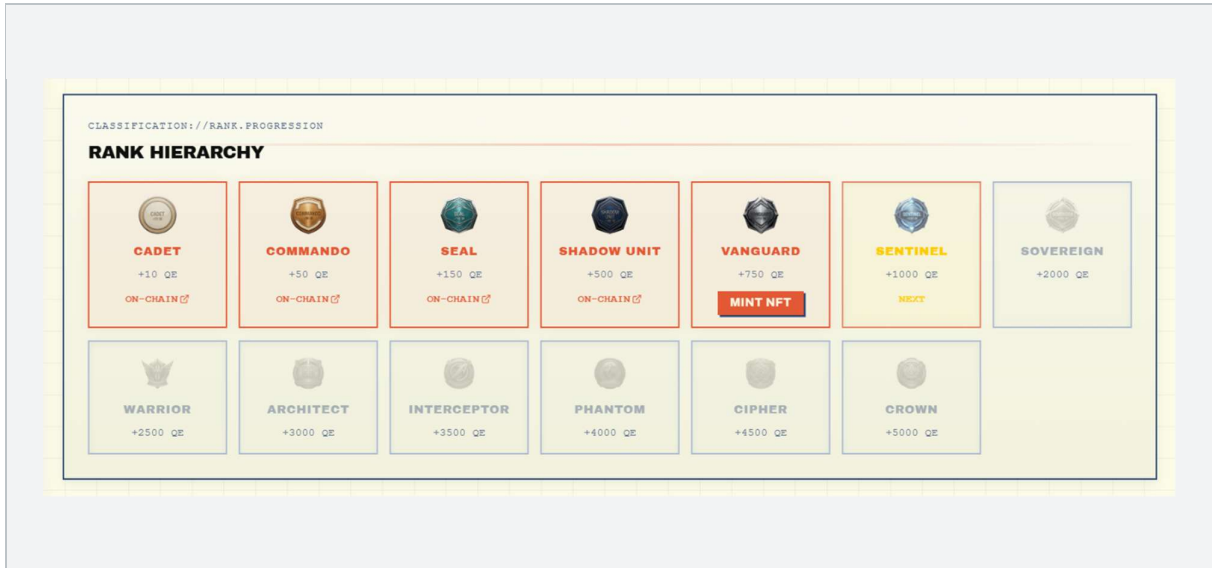
0x383c84d63cf906330e45ce365bfd3599352909469d8f05eafeec4da1fece2f29

Supporting Evidence — Screenshots

The following screenshots provide visual proof of the state desynchronization. The contrast between explorer state and frontend state is the core evidence of this report.

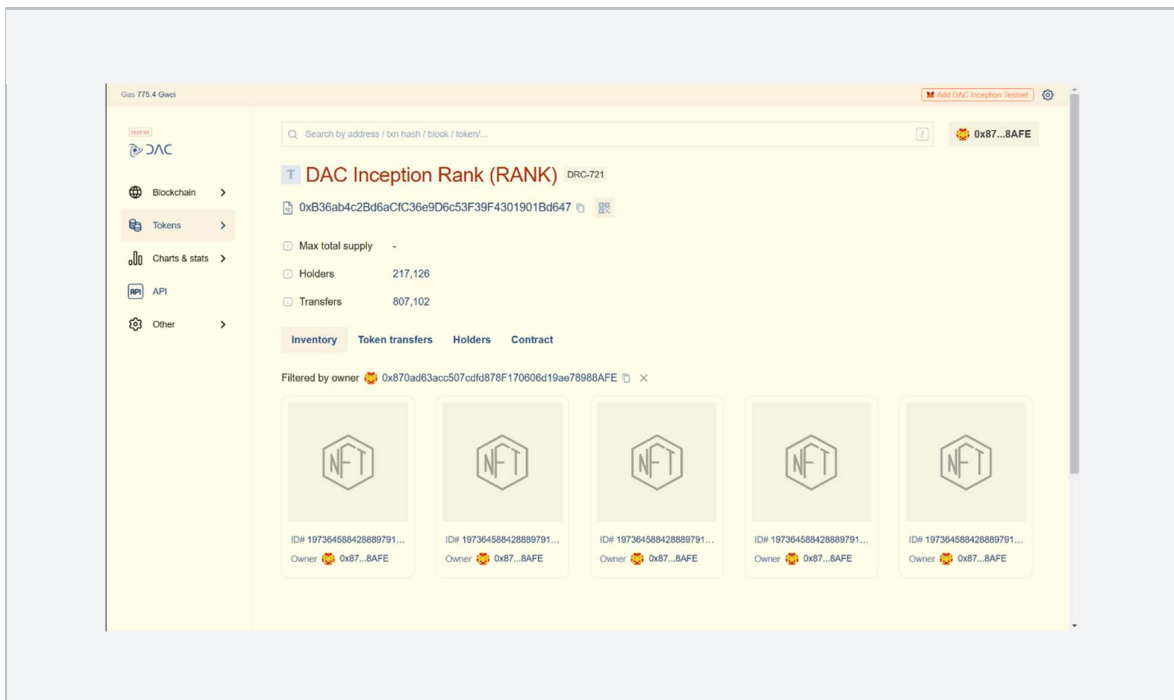
Screenshot 1 — Web Dashboard: Badge Hierarchy State

Shows the DAC Inception badge hierarchy. Other badges correctly display ON-CHAIN status with explorer links. One badge remains incorrectly marked as ready to mint despite being confirmed on-chain.



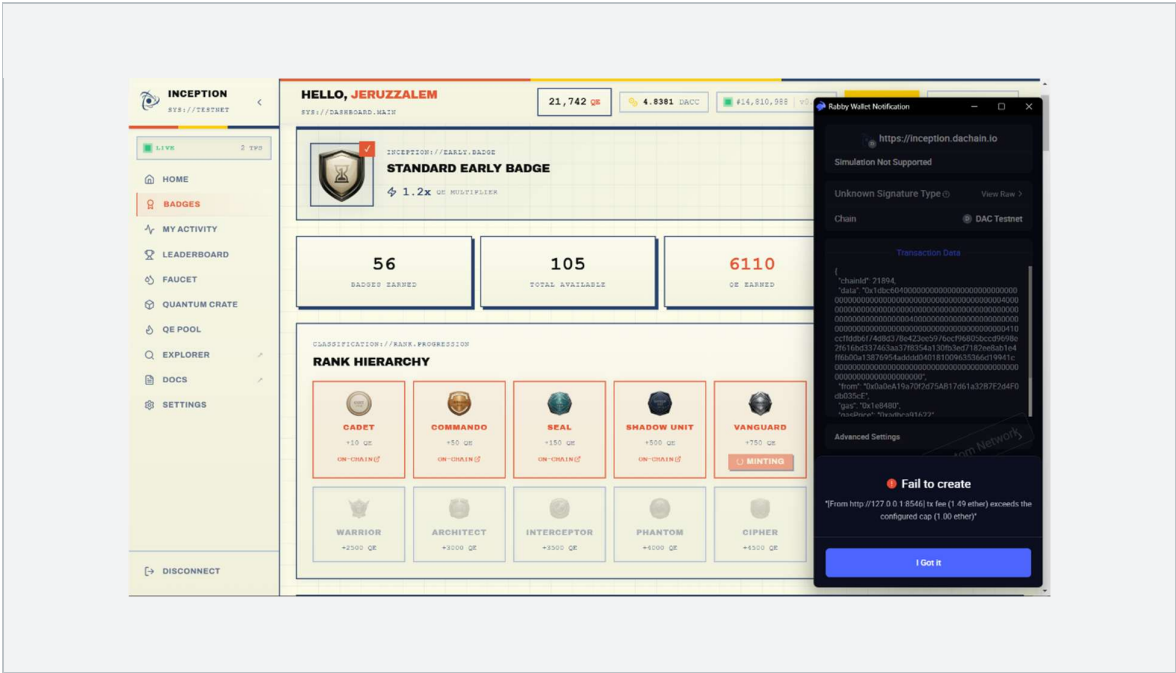
Screenshot 2 — Explorer: NFT Confirmed On-Chain

DAC blockchain explorer view of address 0x870ad6...AFE showing NFT ownership confirmed. 5 NFTs present in wallet — verifying successful on-chain mint.



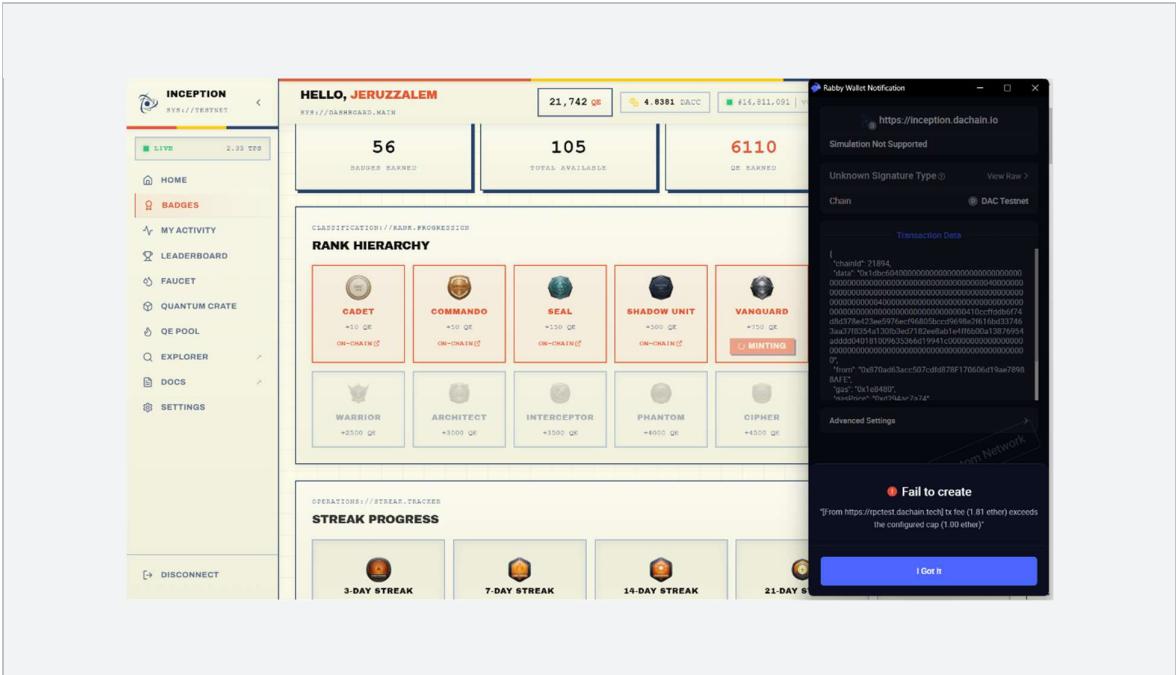
Screenshot 3 — Mint Process via Local RPC (127.0.0.1:8546)

Mint transaction flow using self-hosted local RPC. Shows Rabby Wallet interaction and transaction submission through the WSL node endpoint.



Screenshot 4 — Mint Process via Official RPC (rpctest.dachain.tech)

Mint transaction flow using the official public RPC endpoint as comparison. Confirms the desynchronization issue is not RPC-dependent — frontend state remains stale regardless of RPC source.



Technical Analysis — Possible Root Causes

The blockchain layer operated correctly and finalized the mint transaction. The desynchronization exists at the application layer. The following root causes are identified in order of likelihood:

- **Indexer Delay** — The frontend depends on a backend indexer rather than querying contract state directly. If the indexer has not processed the mint event, the UI reflects stale pre-mint state regardless of on-chain reality.
- **Event Listener Failure** — The mint event emitted by the NFT contract may not have been captured or processed correctly by the backend event listener, leaving the indexer database unchanged.
- **Cached Session State** — The frontend wallet session or browser cache may retain stale badge status from before the mint, not re-querying the backend on page load.
- **RPC Read Inconsistency** — The RPC endpoint serving the frontend may be lagging behind the canonical chain state, returning outdated contract reads for badge ownership checks.

Note on `eth_estimateGas` Revert

The `eth_estimateGas` execution revert observed in the node log is a separate and secondary issue. The custom error selector `0xf3998cd1` returned in `errdata` is a Solidity custom error — likely a precondition check within the NFT contract that fires during simulation (e.g. ownership guard, duplicate mint check, or signature validation). This does not indicate transaction failure. The transaction was submitted and finalized successfully despite simulation failure.

User Impact

- Users may incorrectly conclude that minting has failed, despite successful on-chain execution.
- Users may repeatedly attempt to mint, triggering unnecessary transaction fees and potential contract-level rejections.
- Dashboard progression and badge hierarchy display become visually inconsistent with actual on-chain state.
- User confidence in the mint system reliability may degrade due to UI inaccuracy.
- Risk of double-mint attempts if the stale UI remains active long enough for users to resubmit.

Recommendations

For DAC Team — Application Layer

- Implement direct on-chain contract state verification as fallback after mint completion, independent of indexer state.
- Add transaction receipt validation before frontend state rendering — UI should update on TX confirmation, not indexer confirmation.
- Display a pending synchronization indicator (e.g. "Confirming on-chain...") instead of retaining the stale mint button.
- Add a manual refresh or re-sync mechanism in the dashboard UI for users to trigger a state update.
- Improve indexer monitoring and event synchronization observability — alert on processing lag.

For Testnet Participants

- Verify mint success via the DAC blockchain explorer before attempting any follow-up mint action.
- Perform a hard refresh (Ctrl + Shift + R) and disconnect/reconnect wallet if UI appears stale.
- Do not re-attempt minting if the explorer confirms the NFT is already present in your wallet.

Conclusion

Layer	Status
Blockchain / on-chain layer	Operational — TX finalized, NFT confirmed
Smart contract execution	Successful — mint completed despite estimateGas simulation failure
Backend indexer / event sync	Delayed or incomplete — mint event not reflected
Frontend UI state	Desynchronized — stale state persists post-mint

The DAC blockchain layer operated correctly throughout this investigation. The mint transaction was finalized on-chain, NFT ownership was confirmed via explorer, and node logs verified successful submission. The inconsistency exists entirely at the application layer, where frontend state failed to synchronize with verified on-chain data. Current findings suggest delayed or incomplete indexer synchronization between DAC backend infrastructure and dashboard rendering systems.